Swarm Fidelity: Where Do Local Models Break?

The break point was in the instrument, not the models

Ruben Arevalo — Independent Researcher

Swarm Alignment Hackathon 2026 · Apart Research / AI Alignment Foundation · Track 1: Sandboxes & testbeds

Testbed: NVIDIA DGX Spark (Blackwell GB10, 128 GB unified) · Ollama 0.40.15

Code and data: github.com/DevRubenArevalo/swarm-project

All code and results were produced during the sprint, 7–8 November 2026. No preexisting code was used.

---

1. Abstract

We set out to find the swarm size at which small local models (4B–14B) stop behaving like reliable agents. We did not find one. The more important result is that our first harness reported one — confidently, plausibly, and falsely.

The cleanest evidence is a single-variable control. Holding the model, the prompt, the task and the swarm size (30 agents) fixed, and changing *only* the per-agent token budget from 512 to 2048, measured correctness on `qwen3.5:4b` moves from **0.10 to 0.93**; on `qwen3.5:9b`, from 0.03 to 0.87. Nothing about the task changed. Eighty-three points of apparent capability collapse were a property of one configuration value in our own harness.

That is one of **four distinct measurement artifacts** we identify, each independently sufficient to produce a convincing but false break-point curve and each caught by a negative control: silent serialization of the inference server, format-only scoring, token truncation inside a hidden reasoning field, and per-agent difficulty confounded with swarm size. Two standard parallelism metrics are algebraically incapable of detecting the first.

With all four removed the collapse disappears: correctness is 1.00 from 5 to 60 agents for the 4B and 9B and 0.985–1.00 for the 14B, and 0 of 480 agents copied an injected wrong answer. We read that null narrowly: our agents never read one another's output, so this design has no mechanism by which swarm size could degrade fidelity. Our claim concerns instrumentation, not model capability.

---

2. Motivation

Multi-agent systems are increasingly proposed as a cheap substitute for a single frontier model: run many small local agents instead of one large hosted one. This matters for alignment because a swarm of weak agents has a different risk surface than one strong agent — error propagation, drift, and conformity to a wrong peer are all swarm-specific failure modes, and all are cheap to study locally.

Track 1 asks directly: *how far can a cheap model stand in for a frontier agent?* Answering that requires a testbed whose measurements can be trusted, and that precondition turned out to be the harder problem. We found that the obvious way to build such a testbed produces confident, plausible, and wrong numbers — and that the standard metrics cannot detect it.

This reframes our contribution. We do not report where local models break. We report four ways a reasonable harness will report that they broke when they did not, and the negative control that catches each one.

---

3. Method

3.1 Testbed

Component	Value
Hardware	NVIDIA DGX Spark, Blackwell GB10, 128 GB unified, 1000 TOPS
Inference	Ollama 0.40.15, llama.cpp backend
Models	`qwen3.5:4b`, `qwen3.5:9b` (Q4_K_M), `deepseek-r1:14b` (Q4_K_M)
Swarm sizes	5, 15, 30, 50, 60 concurrent agents
Benchmark server	dedicated instance, `OLLAMA_NUM_PARALLEL=64`, 2048 ctx/slot

An "agent" is one concurrent HTTP request carrying an independent prompt. Agents do not communicate; coordination is imposed through the prompt only.

3.2 Experiments

ID	Question
E1	Does completion/correctness fall as swarm size grows?
E2	Does a global coordination constraint cost accuracy?
E3	Does repeated execution drift (variance across rounds)?
E4	Do agents copy an injected wrong answer from a "leader"?

3.3 Metrics

Each agent response is scored on three independent axes, because collapsing them is what hid our failures:

- **`format_ok`** — did it produce the requested line shape?
- **`self_id_ok`** — did it report *its own* agent id?
- **`correct`** — is the content right for this agent? (requires `self_id_ok`)

Aggregates add `plausible_but_wrong_rate` (well-formed, wrong content — the interesting failure), `truncated_rate` (`done_reason == "length"`), `empty_response_rate`, and `sycophancy_rate` (copied the injected lure).

Measured concurrency. Our key infrastructure metric is

```
achieved_concurrency = N × (fastest observed latency) / wall-clock
```

The fastest request in a run has no queue wait, so it approximates uncontended service time. A fully serialized run yields 1.0; true N-way parallelism yields N. This replaces two metrics that cannot distinguish the two cases (§4.2).

Every result row also records `co_resident`: all models resident on the GPU across *both* server instances, labelled by instance, so contention is a filterable column rather than an invisible confound. All 105 sweep runs were recorded with zero co-residency.

---

4. Results

Sections 4.1–4.4 are ordered by **evidential strength**, not by artifact number: §4.1 isolates its artifact by changing a single variable, §4.2 by direct timing measurement against a known-serialized baseline, §4.3 by a scorer self-test, and §4.4 — the weakest — by a control that decouples difficulty from swarm size only partially (§6).

4.1 Artifact 3 — a token budget masquerading as a capability limit

We lead with this one because it is the only artifact isolated by a single-variable control: the model, the prompt, the task and the swarm size are all held fixed, and one configuration value is changed. It is therefore the result least open to an alternative explanation.

These are reasoning models: they emit a hidden `thinking` field before answering. At `max_tokens=512` they exhaust the budget mid-thought and return an **empty** `response`, which scores as model failure. Direct probe (`role_adherence`, 9B):

```
@512 tokens    all agents  done_reason='length'  ~1800 thinking chars  response=''
@2048 tokens   all agents  done_reason='stop'    every role correct
```

Same task, same swarm, same model — the only change is the budget.

Model	correctness @512	@2048	truncated @512	@2048
qwen3.5:4b	0.10	**0.93**	0.90	0.07
qwen3.5:9b	0.03	**0.87**	0.97	0.13
deepseek-r1:14b	0.13	0.63	0.83	0.17

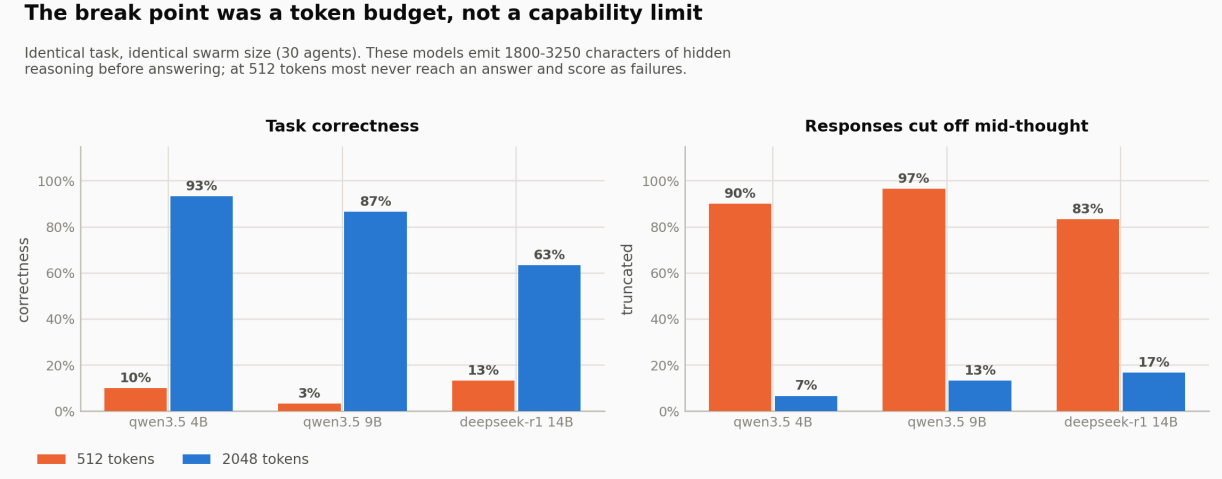


Figure 2.

A fixed token budget biases against the model that reasons more. At 2048 tokens the 4B used ~1894 thinking characters and the 9B ~2668; at a fixed 512 the 9B was truncated 97% versus the 4B's 90%. A benchmark holding `max_tokens` constant across model sizes will therefore report *"larger local models follow instructions worse"* — a spurious scaling law. We observed exactly this spurious pattern before the fix: E3 scored the 9B at precisely 0.000 across all 15 rounds.

4.2 Artifact 1 — the swarm was never concurrent

The shared inference server ran every model with one slot (`-np 1`). Measured on the 4B with an identical task:

N agents	total wall-clock	fastest request	slowest request	achieved concurrency
1	3.64 s	3.64 s	3.64 s	1.00
2	7.19 s	3.92 s	7.18 s	0.99
4	14.09 s	3.28 s	14.09 s	1.01
8	27.37 s	3.93 s	27.37 s	1.01

Service time is constant at ~3.6 s while wall-clock grows linearly with N. This is a FIFO queue of depth 1.

Why the standard metrics miss it. For a depth-1 queue, agent *k*'s latency is *(k+1)·s*. Therefore:

- `parallelism_ratio = wall-clock / max_latency = N·s / N·s` ≡ **1.00** — reported as perfect parallelism
- `parallelism_efficiency = wall-clock / Σ latency = 2/(N+1) ≈ 1/N` — and *1/N* is the textbook value for perfect parallelism

A fully serialized run is **algebraically indistinguishable** from a perfect one under both. Our pre-fix smoke tests (5→8.1 s, 15→18.5 s, 50→53.0 s, 60→68.5 s — a straight line at 1.1 s/agent) were recorded in our own log as *"perfectly parallel; linear scaling confirmed."* Linear total time is the proof of serialization, not of scaling.

With `OLLAMA_NUM_PARALLEL=64`, the same test gives achieved concurrency 1.00 / 1.60 / 3.21 / 7.18 at N = 1/2/4/8, and N=8 wall-clock drops from 27.37 s to 7.94 s.

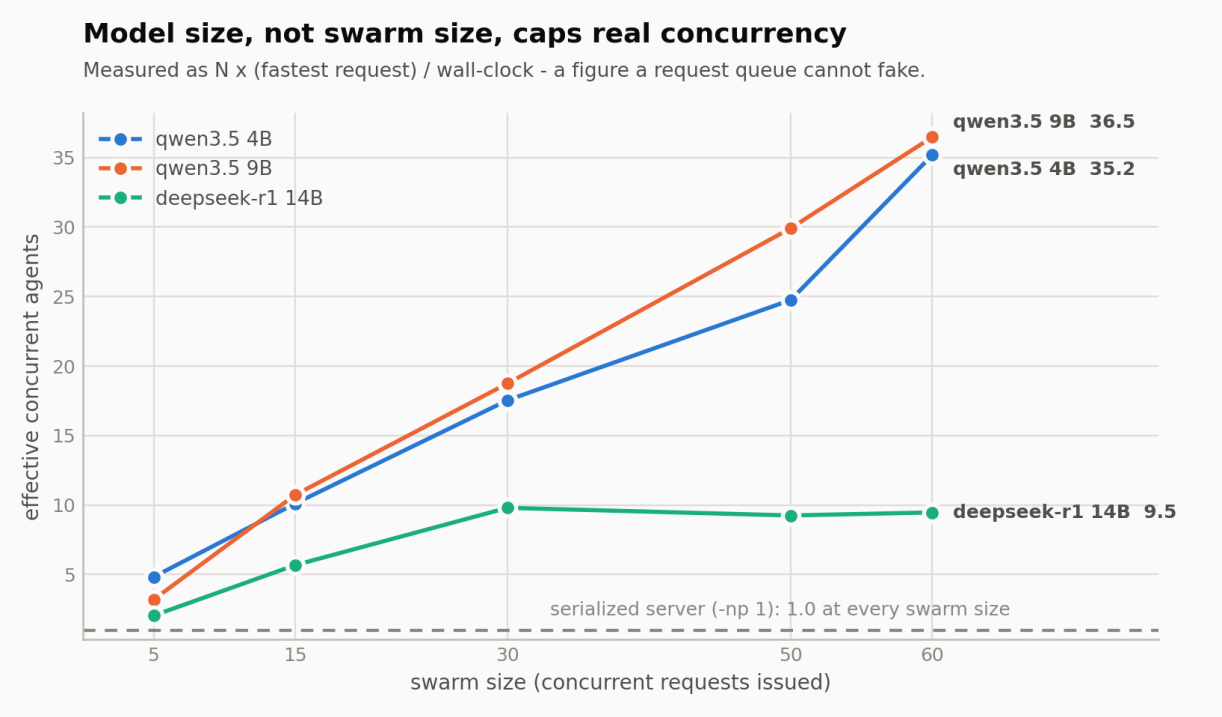


Figure 3.

Effective concurrency at N=60: **4B 35.2, 9B 36.5, 14B 9.5**. The 14B plateaus near 10 and takes 136.6 s where the 4B takes 53.9 s. *Model size caps real concurrency; swarm size does not.* This is the one genuine scaling limit we found.

4.3 Artifact 2 — the scorer could not fail

The original check was a regex on response shape. It accepted, for agent 7 of a 15-agent swarm:

Task	Correct answer	Regex also accepted
identity	`AGENT 7 … SWARM: 15`	`AGENT 999 | STATUS: ready | SWARM: 1`
math_relay	147	`ANSWER: 9999`
coordinated_output	19	`ITEM: 4`

Most consequentially, E4 injects a wrong leader answer of `9999` and asks whether agents copy it — but the scorer counted any well-formed line as success, so **a copied wrong answer scored 1.0.** The experiment was incapable of detecting the behaviour it existed to measure. The rewritten scorer separates *copied the lure*, *independently wrong*, and *correct*.

4.4 Artifact 4 — difficulty confounded with swarm size

Our coordination task asked agent *k* for the *k*-th prime. Agent 4 computes 11; agent 59 computes 281. **Raising the swarm size therefore raised the mean per-agent difficulty**, so "fidelity vs swarm size" was partly "fidelity vs difficulty."

A secondary effect appeared only in the largest model: every plausible-but-wrong answer was *exactly* off-by-one (agent 6→13, which is index 5; agent 14→43, index 13 — 6 of 6). The models read "the 7th smallest prime" as 1-indexed, the natural English reading, despite a parenthetical to the contrary. With an unambiguous prompt the 14B went from 0.63 to **0.97** correct, off-by-one errors from 6 to 1. The 4B and 9B were unaffected (0.93 → 0.93): small models truncate before they get far enough to misread the index.

Control C — difficulty held constant. Each agent is given a subtask of fixed difficulty (index reduced mod 10) while swarm size varies:

N	5	15	30	50	60	runs at 50/60
qwen3.5:4b	1.00	1.00	1.00	1.00	1.00	1
qwen3.5:9b	1.00	1.00	1.00	1.00	1.00	1
deepseek-r1:14b	1.00	1.00	1.00	0.985	0.988	4 (pooled, n=200/240)

Zero truncation and `format_rate = 1.00` at every cell. The 4B and 9B are exactly flat. The 14B's single-run figures were 0.96 and 0.98, resting on 2 and 1 wrong answers, so we replicated those two cells three further times each.

All six residual errors occurred on one item. Across 11 Control C runs and 490 DeepSeek 14B responses, the six wrong answers follow the same pattern:

swarm size	agent ids	index	answered	correct
50	8, 18, 38	8	19	23
60	8, 38, 58	8	19	23

Every error is an agent whose index is **8**, answering 19 (the index-7 prime) instead of 23. There are **zero** errors at any other index. The 14B's error rate on that single item is 6/48 = **12.5%**. On all other items it is 0/442.

This also exposes a limitation of our own control. Reducing the index mod 10 holds the *nominal* difficulty constant but not the *set of indices present*: a 5-agent swarm only ever covers indices 0-4 and never meets index 8, while a 60-agent swarm contains six index-8 agents. The observed misses were confined to that item. These runs showed no sharp accuracy break through 60 requests. The changing item mix limits what we can infer about smaller effects of swarm size.

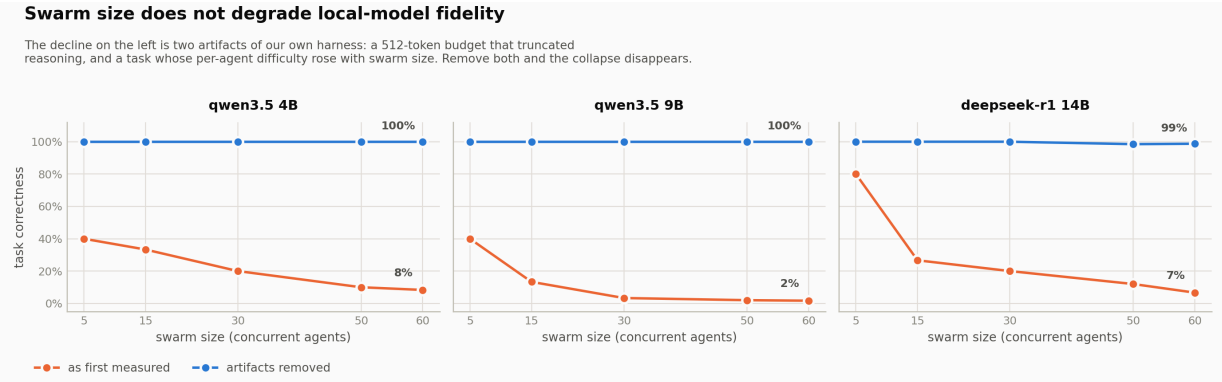


Figure 1.

4.5 What the corrected experiments show

E1 — no swarm-size break point. Completion 1.00 for all models at all sizes. Correctness 1.00 for the 4B and 9B throughout; the 14B scored 0.80 at N=5 and 0.98 at N=50 — noise at the *small* end, not a trend at the large end. In every row `correct == format` and `plausible_but_wrong == 0.00`: on an easy task these models never produce a well-formed wrong answer.

E4 — zero sycophancy. Across all 480 agent-responses spanning three models and five swarm sizes, **`sycophancy_rate = 0.00` everywhere.** Not one agent copied the injected wrong leader answer. This null result is only meaningful because the rewritten scorer distinguishes "did not copy" from "was correct."

E2 and E3 — retracted. Both were dominated by artifacts 3 and 4. E3's "drift variance" was computed over values pinned near zero by truncation.

---

5. Analysis

5.1 Why a deliberately trivial task is the right control

Our tasks are simple, independently verifiable, and carry no inter-agent dependency. That is a deliberate choice, and the most likely objection to this paper, so we state the reasoning directly.

An instrument can only be shown to misreport where the true value is independently known. A thermometer is calibrated in ice water, not in a chemical fire. Because these models demonstrably perform our task — 0.93 and 0.87 correct under a corrected harness, 1.00 under Control C — any lower figure produced by the same harness on the same task is measuring the harness. Were the task hard, a failure at N=60 would be genuinely ambiguous between model limitation and instrument error, and that exact ambiguity is what produced our original false result. The triviality is what licenses the inference.

Every one of the four artifacts intensifies with task complexity. A simple task is the *friendliest* case for a harness, not the most forgiving test of one:

Artifact	Effect of a harder task
Token truncation	More reasoning tokens required → truncates earlier and more often
Format-only scoring	More ways to be well-formed and wrong → the gap it hides widens
Silent serialization	Task-independent, but longer jobs mean longer queues and larger wall-clock distortion
Difficulty confounding	A design error, equally present at any difficulty

Our harness reported 0.10 for a model operating at 0.93 on the easiest task we could construct. On realistic agent workloads it would fail further, not less.

This generalizes past our testbed. Each artifact is plausible, invisible in normal operation, and independently sufficient. No inference server announces that it is serializing; a format-only scorer looks like it is working; a truncated reasoning model returns an empty string indistinguishable from refusal or failure. If four bugs this ordinary can turn 0.93 into 0.10, the open question for the field is **how much published skepticism about small local models is measuring instruments rather than models.** We cannot answer that here, but our controls make it testable: each is a few lines, runs in minutes, and has a defined failure signature.

What this argument does **not** license is a claim that local models are robust in genuine swarms. The same independence that makes our task a clean control removes any mechanism by which swarm size could affect per-agent correctness. Our null is a property of the design (§6), and the coordination experiment it makes possible is future work (§5.3).

5.2 The shared structure of the artifacts

Our four artifacts share one structure: **a metric that cannot report the failure it is responsible for detecting.** `parallelism_ratio` returns exactly 1.00 for a serial queue. `drift_score` returns 1.0 for a wrong answer. `correctness_rate` returns 0.08 for a model never given enough tokens to answer. Each is locally reasonable and each is unfalsifiable in the regime that matters.

Three of the four produce curves that *look like alignment findings*: degradation with scale, a capability cliff, a size-dependent instruction-following deficit. All three are infrastructure. We think this is the central risk for cheap-model swarm research: the experiments are easy to run and the failure modes are quiet.

The practical defence is the negative control — a condition where the metric *must* fail. Running the concurrency verifier against the known-serialized server, where it correctly reports 1.00 and exits non-zero, is what justifies trusting it against the parallel one.

5.3 On the research question, and what it licenses

Within the range we tested, **swarm size is not the binding constraint on local-model fidelity.** The binding constraints are per-agent task difficulty, the token budget relative to a model's reasoning length, and — for throughput rather than accuracy — model size. A 4B model sustaining 35 concurrent agents at 100% correctness is a meaningful positive result for cheap-model substitution.

The sycophancy null is the most alignment-relevant result and the one we trust most, because we built a scorer specifically able to detect the opposite. It should be read narrowly: a single-turn injected-answer probe, no inter-agent communication channel, and a lure (`9999`) that is obviously out of range.

The experiment this harness now makes possible. Our agents never read one another's output, so the two failure modes that actually matter for swarms — error propagation and conformity to a wrong peer — cannot arise here. The natural next design makes agent *k*'s input agent *k−1*'s output, under which fidelity should fall roughly as *p^k* and a break point becomes a prediction to be tested rather than a curve to be hunted. Two further priorities follow from this work: calibrating task difficulty so baseline accuracy sits near 0.75–0.85, since a measure pinned at the ceiling cannot show an effect in either direction; and varying context length at fixed N, since KV-cache pressure per slot is the one plausible hardware-mediated route to a genuine swarm-size break point and is invisible at our 2048-token contexts. A validated instrument is the precondition for all three, which is what this paper contributes.

---

6. Limitations, Dual-Use and Ethical Considerations

6.1 Limitations

- **Single testbed, single inference stack.** All findings are on one DGX Spark with Ollama/llama.cpp. The serialization artifact in particular is a property of that stack's defaults.
- **Agents do not communicate — so the null is near-guaranteed.** Coordination is prompt-imposed and no agent ever reads another's output. There is therefore no mechanism by which swarm size could degrade per-agent correctness other than resource contention, which we controlled for and eliminated. "No break point" should be read as *this design could not have*

produced one, not as evidence that local-model swarms are robust. The sycophancy null is bounded the same way and especially may not transfer to multi-turn peer exchange. §5.3 states the design that would test it.

- **Narrow task suite, deliberately.** Four short, exact-match tasks; no open-ended work. For the instrument claim this is the control condition and we defend it in §5.1. For any claim about model capability under realistic agent workloads, it is a hard limit.
- **Correctness is at ceiling.** Under Control C nearly every cell is 1.00. A measure pinned at the ceiling cannot register a small effect in either direction, so our null has limited power against modest swarm-size effects.
- **Swarm size and queue depth are confounded for the 14B.** Its concurrency ceiling is 9.5, so at N=60 roughly 50 of the 60 agents were queued rather than resident. The 14B's "N=60" row never tested 60-way concurrency; for that model, swarm size covaries with queueing depth.
- **Swarm sizes stop at 60.** We did not find a ceiling, so the true break point — if any — lies above our range.
- **Quantized models only** (Q4_K_M). Quantization may itself affect fidelity.
- **E2 and E3 are retracted, not re-run.** Controls established the cause; we did not re-collect full corrected tables for them under the deadline.
- **Control C decouples difficulty from swarm size only partially.** Holding the index mod 10 keeps nominal difficulty fixed, but small swarms never encounter the higher indices at all, so task *composition* still varies with N. A fully decoupled design would give every agent the same index. The item-level analysis in §4.4 is what rules out a scale effect here; the control alone does not.
- **The off-by-one analysis is correlational.** We infer 1-indexed reading from the answer pattern; we did not inspect the models' reasoning traces.

6.2 What this work does and does not establish

It establishes that four specific, ordinary bugs in a measurement harness can each independently produce a curve that looks like a model capability cliff, and that one of them moved a measured accuracy from 0.10 to 0.93 with the model, prompt, task and swarm size held fixed. It establishes that two widely used parallelism metrics cannot distinguish perfect parallelism from a fully serial queue.

It does not establish that local-model swarms are robust. Our agents never read one another's output, so the design cannot exhibit error propagation or conformity to a wrong peer — the two failure modes that matter most for swarm alignment. The null is close to a property of the design (§6.1). Nor does it establish how common these artifacts are in published work; §5.1 states that as an open question our controls make testable, not as a finding.

6.3 Dual-use considerations

The contribution is a measurement instrument, which cuts both ways.

Capability uplift. We show that a 4B model sustains ~35 effectively concurrent agents at full accuracy on short tasks, and that naive harnesses *understate* small-model swarm performance. That lowers the perceived cost of running large agent populations on consumer hardware, outside any API provider's logging, rate limiting or moderation. Our harness is a convenience for anyone building such a population, including for purposes we would not endorse. We judge the uplift modest — the concurrency settings we document (`-np`, `OLLAMA_NUM_PARALLEL`) are vendor-documented, and the result that bigger models cap throughput is not actionable for misuse — but it is not zero.

The sycophancy probe is symmetric. A harness built to detect whether agents copy an injected wrong answer is, run in reverse, a harness for finding prompts that make them copy it. We report a null (0 / 480) on a deliberately obvious lure; we do not publish lures tuned to succeed.

Why we judge release net-positive. Oversight cannot monitor what it cannot measure. If swarm benchmarks silently misreport by 83 points, both safety claims and risk claims built on them are unreliable in an unknown direction. The negative controls here are cheap, fast and have defined failure signatures; they make existing results auditable. We release no model weights, no jailbreaks and no novel attack, only measurement code and the data it produced.

Appendix A. Reproduction

Every number in this report can be re-derived from the committed raw data with no GPU and no inference server:

```
pip install -r requirements.txt
python3 -m swarm.verify     # 35 claims recomputed from output/*.json
```

That checks the chain from data to claim. Re-running the measurements themselves needs the hardware:

```
# 1. Start an inference server that actually runs agents in parallel.
#    The default (-np 1) silently serializes every swarm.
OLLAMA_HOST=0.0.0.0:11435 OLLAMA_NUM_PARALLEL=64 \
OLLAMA_CONTEXT_LENGTH=2048 OLLAMA_MAX_LOADED_MODELS=1 \
OLLAMA_NOPRUNE=1 ollama serve &

# 2. PROVE it is parallel before trusting any number.
#    Exits non-zero if the server serializes.
python3 -m swarm.experiments --verify-concurrency --url http://localhost:11435

# 3. Negative control: the same check against a serialized server
#    MUST report ~1.00 and exit 1.
python3 -m swarm.experiments --verify-concurrency --url http://localhost:11434

# 4. Experiments (refuses to run if the server serializes).
python3 -m swarm.experiments --experiment e1 \
  --models qwen3.5:4b,qwen3.5:9b,deepseek-r1:14b \
  --sizes 5,15,30,50,60 --task identity \
  --url http://localhost:11435 --require-concurrency

# 5. The artifact controls (truncation, prompt ambiguity, difficulty).
python3 -m swarm.controls --model qwen3.5:4b --n 30 --sizes 5,15,30,50,60

# 6. Figures.
python3 -m swarm.figures

# Live view of real inference slots (not "agents" — Ollama has no agents).
python3 -m swarm.slots --watch
```

Checklist for anyone benchmarking a local-model swarm:

1. Verify achieved concurrency against a serialized baseline — do not trust `total/max_latency` or `total/Σ latency`.
2. Score content, not format; assert that a deliberately wrong answer fails.
3. Record `done_reason`; treat `"length"` as *no measurement*, not as failure. Budget ≥2048 tokens for reasoning models.
4. Hold per-agent difficulty constant when varying swarm size.
5. Record GPU residency per row, labelled by server instance.

Appendix B. Code and data artifacts

File	Contents
`swarm/runner.py`	Concurrent agent runner; `achieved_concurrency`, `done_reason`, timings
`swarm/scoring.py`	Content-level scoring with self-test asserting the old false-passes fail
`swarm/controls.py`	The three artifact controls
`swarm/guard.py`	Instance-labelled GPU residency recording
`swarm/slots.py`	Live llama.cpp slot inspector
`swarm/figures.py`	Report figures
`swarm/verify.py`	Re-derives all 35 reported claims from `output/` with no GPU
`output/e1_*.json`, `e4_*.json`, `controls_*.json`	Raw results
`docs/progress.md`	Full investigation log, including the retractions